

Universidade de Vigo

**ESCOLA SUPERIOR DE ENXEÑARÍA INFORMÁTICA**

Memoria do Traballo de Fin de Grao que presenta

**D. Nome Alumna/o**

para a obtención do Título de Graduado en Enxeñaría Informática

**Título do Traballo de Fin de Grado**



Xullo, 20XX

**Traballo de Fin de Grao N°:**

**Titor/a:**

**Área de coñecemento:** Linguaxes e Sistemas Informáticos

**Departamento:** Informática

## Agradecimientos

[Texto]

## Dedicatoria

[Texto]

# Índice

|                                                                                |           |
|--------------------------------------------------------------------------------|-----------|
| <b>1. Introducción</b>                                                         | <b>5</b>  |
| <b>2. Objetivos</b>                                                            | <b>6</b>  |
| 2.1. Objetivo principal . . . . .                                              | 6         |
| 2.2. Objetivos específicos . . . . .                                           | 6         |
| 2.3. Alcance y limitaciones . . . . .                                          | 6         |
| <b>3. Antecedentes y contexto</b>                                              | <b>7</b>  |
| <b>4. Resumen de la solución propuesta</b>                                     | <b>9</b>  |
| <b>5. Marco teórico y práctico. Herramientas empleadas</b>                     | <b>10</b> |
| 5.1. Marco teórico . . . . .                                                   | 10        |
| 5.1.1. Entorno: Reglas del ajedrez 5x5 . . . . .                               | 10        |
| 5.1.2. Arquitectura basada en Transformers: Encoder y Self-Attention . . . . . | 12        |
| 5.1.3. Modelado del Aprendizaje por Refuerzo . . . . .                         | 13        |
| 5.1.4. Diseño Experimental e Hipótesis . . . . .                               | 14        |
| 5.2. Marco práctico . . . . .                                                  | 14        |
| 5.2.1. Codificación del Espacio de Estados (Input) . . . . .                   | 14        |
| 5.2.2. Codificación del Espacio de Acciones (Output) . . . . .                 | 14        |
| 5.2.3. Arquitectura Transformer . . . . .                                      | 15        |
| 5.2.4. Algoritmo RL y Búsqueda en Árbol Monte Carlo (MCTS) . . . . .           | 16        |
| 5.2.5. Herramientas y tecnologías . . . . .                                    | 17        |
| <b>6. Planificación y seguimiento</b>                                          | <b>18</b> |
| <b>7. Principales aportaciones</b>                                             | <b>19</b> |
| <b>8. Conclusiones</b>                                                         | <b>19</b> |
| <b>9. Vías de trabajo futuro</b>                                               | <b>19</b> |
| <b>Referencias</b>                                                             | <b>20</b> |
| <b>A. Anexos</b>                                                               | <b>21</b> |

## Índice de figuras

1. Posición inicial del ajedrez Gardner. . . . . 11
2. Descripción del experimento y métricas obtenidas. . . . . 19

## Índice de cadros

## 1. Introducción

Este Trabajo de Fin de Grado consiste en la aplicación y estudio del Aprendizaje por Refuerzo (RL) sobre una variante de ajedrez simplificada, el ajedrez  $5 \times 5$  (ajedrez de Gardner o Minichess). El objetivo central es validar el uso de arquitecturas Transformer en tareas de RL, estudiando el efecto de las representaciones del tablero y comprobando si el preentrenamiento supervisado mejora la convergencia del agente.

Dado que el ajedrez  $8 \times 8$  demanda una capacidad computacional y un tiempo de ejecución que excede con creces los recursos disponibles en el marco del trabajo, se ha optado por utilizar el ajedrez  $5 \times 5$ . Este entorno reducido mantiene las dinámicas esenciales del aprendizaje por refuerzo adversarial, como las acciones legales, las recompensas diferidas y la planificación estratégica, pero se mantiene computacionalmente manejable para la experimentación ágil.

El resto de este documento se estructura de la siguiente manera: en la Sección 2 se detallan los objetivos generales y específicos junto con el alcance del trabajo; en la Sección 3 se describe el estado del arte y los antecedentes del problema; en la Sección 4 se resume la solución metodológica planteada; en la Sección 5 se desarrolla el marco teórico y práctico, justificando las herramientas y el modelado matemático; en la Sección 6 se muestra la planificación temporal del proyecto; y las secciones finales presentan las aportaciones principales, conclusiones y futuras líneas de investigación.

## 2. Objetivos

### 2.1. Objetivo principal

Diseñar, entrenar y evaluar un agente de Inteligencia Artificial basado en arquitecturas Transformer mediante Aprendizaje por Refuerzo para el entorno de ajedrez  $5 \times 5$ .

### 2.2. Objetivos específicos

1. Implementar un algoritmo de RL para ajedrez  $5 \times 5$ , junto a la lógica necesaria para self-play.
2. Desarrollar y adaptar una arquitectura Transformer para procesar estados del tablero y predecir distribuciones de probabilidad sobre el espacio de acciones.
3. Evaluar empíricamente el impacto de distintas representaciones del tablero y del espacio de acciones en la velocidad de aprendizaje y rendimiento del modelo.
4. Comparar el rendimiento y la eficiencia de convergencia entre un agente entrenado mediante RL *tabula rasa* frente a un agente con inicialización mediante aprendizaje supervisado.

### 2.3. Alcance y limitaciones

El alcance de este trabajo se ha enmarcado en los siguientes límites prácticos e investigativos:

- **Entorno de juego:** Se limita estrictamente a la variante Gardner de ajedrez  $5 \times 5$ . No se explorarán otras variantes del miniajedrez ni el ajedrez estándar  $8 \times 8$ .
- **Recursos computacionales:** El entrenamiento de los modelos se realizará utilizando hardware de consumo accesible (una GPU de gama media/alta) con un límite temporal de entrenamiento acotado a un número de horas que permita la experimentación iterativa ágil.
- **Algoritmos de aprendizaje:** Se prioriza la combinación de búsqueda en árbol Monte Carlo (MCTS) con métodos de optimización basados en gradiente de política (estilo PPO / AlphaZero), sin evaluar algoritmos de RL puramente libres de modelo.

### 3. Antecedentes y contexto

El Aprendizaje por Refuerzo (RL) ha demostrado en los últimos años resultados extraordinarios en numerosos campos como el Go, el ajedrez, la síntesis de proteínas, los modelos de lenguaje y la robótica. Con la enorme capacidad de computación actual, que sigue creciendo día a día, el RL aprovecha las características enunciadas en *The Bitter Lesson* de Sutton [1]: la búsqueda masiva y el aprendizaje de funciones a través del muestreo superan a largo plazo a los enfoques basados en conocimiento humano inyectado manualmente.

Ambos procesos, búsqueda y aprendizaje, son altamente exigentes en términos de computación, pero bajo la suposición de que esta aumenta al ritmo de la Ley de Moore, los métodos que aprovechan la computación superan eventualmente aquellos basados en conocimiento humano. Esto es especialmente cierto en entornos (discretos o continuos) con espacios de estado masivos, como son el ajedrez o la robótica.

---

Desde la irrupción del aprendizaje profundo y el aprendizaje por refuerzo en los juegos de tablero, se han explorado diversas arquitecturas neuronales. El hito inicial lo marcó AlphaGo [2], que combinó redes convolucionales (CNNs) con búsqueda de árbol Monte Carlo (MCTS) para vencer al campeón mundial de Go. Posteriormente, esta metodología se generalizó con AlphaZero [3], un algoritmo de RL capaz de dominar el ajedrez, el shogi y el Go partiendo desde cero (*tabula rasa*) y utilizando una única red neuronal para evaluar posiciones y priorizar jugadas. El impacto de estos métodos fué más allá, al demostrar el poder del *deep reinforcement learning* en otros dominios científicos complejos, como la predicción de estructuras de proteínas mediante AlphaFold [4].

Paralelamente, en el ámbito de los motores de ajedrez tradicionales, enfoques como NNUE (*Efficiently Updatable Neural Networks*) [5] tomaron un camino alternativo: en lugar de grandes redes convolucionales, emplean arquitecturas *feed-forward* muy ligeras con sesgos inductivos específicos del ajedrez, optimizadas para ejecutarse en CPU y priorizar la velocidad de evaluación y la profundidad de búsqueda táctica.

En estos motores de juego, las redes neuronales actúan como la parte *estratégica* mientras que la búsqueda actúa como la parte *táctica*, decidiendo a corto plazo bajo una serie de restricciones más claras (jugadas forzadas, amenazas claras como jaques o pérdida de material), y depende más del *cálculo*. Esto es, en cierto modo, reminiscente de cómo los humanos estructuramos nuestro pensamiento a la hora de jugar al ajedrez.

---

Entre los enfoques anteriores, ninguno hacía uso de mecanismos de atención ni de arquitecturas basadas en Transformers, lo que nos lleva a explorar el potencial de estos últimos. Más recientemente, tras el éxito de esta arquitectura en los modelos de lenguaje (LLMs), han surgido aproximaciones que buscan aplicarla a tareas de RL y ajedrez. En este trabajo, se busca demostrar que la arquitectura Transformer es altamente competitiva para la tarea de RL, utilizando un entorno  $5 \times 5$  para facilitar la simulación y el muestreo iterativo.

En este ámbito, Ruoss et al. [6] demostraron la viabilidad de realizar planificación amortizada utilizando Transformers a gran escala aplicados al ajedrez. Por otro lado, Monroe y Chalmers [7] desarrollaron un modelo basado en Transformers para dominar el ajedrez, sentando las bases para Chessformer, una arquitectura unificada propuesta posteriormente por Monroe et

al. [8]. Además, Jenner et al. [9] aportaron evidencia sobre la existencia de capacidades de anticipación y planificación (*look-ahead*) aprendidas de forma implícita por este tipo de redes, mientras que el proyecto open source Leela Chess Zero también ha incorporado y evaluado estas arquitecturas en su motor de juego [10].

Por su parte, el estudio del miniajedrez o ajedrez en tableros reducidos tiene una larga trayectoria. La variante de Gardner fue propuesta originalmente por Martin Gardner en 1969. Este tipo de entornos se ha empleado tradicionalmente para analizar la complejidad matemática del juego y estudiar algoritmos de búsqueda heurística, al reducir drásticamente el espacio de estados sin perder la naturaleza no lineal e interactiva del ajedrez convencional TODO:cita como el paper ese.



## 4. Resumen de la solución propuesta

[TODO: Redactar bien el pipeline]

1. Generación de un dataset de partidas de Minichess con un motor clásico.
2. Estudio, filtrado y procesado de datos.
3. Diseño de arquitectura de un modelo Transformer Encoder.
4. Evaluación de distintos sesgos de input / output en fase de entrenamiento supervisado.
5. Definición del bucle de entrenamiento e implementación de algoritmo de RL (MCTS + PPO).
6. Fase de evaluación contra una *baseline* (ej. Stockfish o heurísticas simples).
7. Evaluación de red preentrenada VS. from-scratch (mismo numero de epochs).

## 5. Marco teórico y práctico. Herramientas empleadas

### 5.1. Marco teórico

#### 5.1.1. Entorno: Reglas del ajedrez 5x5

El ajedrez  $5 \times 5$  en su variante de Gardner es una reducción matemática del ajedrez estándar. Las reglas básicas de movimiento y captura para cada pieza individual son idénticas a las del ajedrez estándar  $8 \times 8$ , con las siguientes modificaciones estructurales:

- **Tablero:** tamaño de  $5 \times 5$  casillas (columnas  $a$  a  $e$ , filas 1 a 5).
- **Disposición inicial:** Cada jugador cuenta con 10 piezas. En la fila 1 (blancas) y fila 5 (negras) se colocan las piezas mayores en el orden: Torre ( $R$ ), Caballo ( $N$ ), Alfil ( $B$ ), Dama ( $Q$ ) y Rey ( $K$ ). Los peones correspondientes se colocan en la fila 2 (blancas) y fila 4 (negras).
- **Omisión de reglas especiales:** No existe el enroque ni el avance doble de peón. Como consecuencia directa, tampoco existe la captura al paso (*en passant*).
- **Promoción de peones:** Cuando un peón alcanza la fila límite opuesta (fila 5 para blancas, fila 1 para negras), es obligatorio promocionarlo a una de las siguientes cuatro piezas: Dama, Torre, Alfil o Caballo.
- **Condiciones de fin de partida:** La partida finaliza por jaque mate, ahogado (*stalemate*), insuficiencia de material para dar jaque mate, si pasan 50 movimientos sin capturas ni avances de peón, o tablas por triple repetición (tres posiciones iguales consecutivas en el tablero).

Esta variante mantiene la naturaleza estratégica y la profundidad del ajedrez, pero en una fracción de su complejidad de cálculo original.

#### Representación de estados mediante notación FEN

Para interactuar con el entorno de ajedrez y alimentar los modelos de aprendizaje, se utiliza la notación FEN (*Forsyth-Edwards Notation*). Esta notación describe una posición particular del tablero mediante una cadena de texto que codifica la ubicación de las piezas, el jugador al que le toca mover y las disponibilidades de reglas especiales. En el contexto del ajedrez 5x5, el FEN se simplifica ya que cierta información no es aplicable a la variante de Gardner (como los enroques o las capturas *en passant*).

El FEN inicial de la variante Gardner se representa como:

```
rnbqk/ppppp/5/PPPPP/RNBQK w - - 0 1
```

La Figura 1 ilustra la disposición física y la colocación de las piezas en el tablero Gardner al inicio de la partida.

#### El ajedrez como juego y proceso teórico de decisión

El ajedrez es un juego de información perfecta y completa. En todo momento los jugadores tienen información total acerca de las funciones de utilidad de su rival y del estado de la partida, tanto presente como pasado. Esto proporciona fundamentos teóricos sólidos para probar algoritmos de RL; un entorno determinista y con funciones de transición bien definidas.



Figura 1: Posición inicial del ajedrez Gardner.

Dado el factor de ramificación estimado del ajedrez estándar (media de 35 hijos por nodo) y la complejidad del árbol de juego ( $10^{120}$ ), propuesta por Claude Shannon en su artículo fundacional [11], la variante de ajedrez  $5 \times 5$  se establece como un entorno matemáticamente equivalente en sus reglas de transición, pero con un espacio de estados acotado.

Para estimar la complejidad de la variante Gardner  $5 \times 5$ :

- **Espacio de estados  $\mathcal{S}$ :** Al colocar un máximo de 20 piezas (10 blancas y 10 negras) sobre 25 casillas, una cota superior combinatoria combinando la colocación de piezas distintas es:

$$|\mathcal{S}_{\text{comb}}| = \frac{25!}{5!5!5!} \approx 7,09 \times 10^{15}$$

Descontando posiciones ilegales (reyes en jaque simultáneo, peones en las filas de inicio/promoción, etc.), el espacio de estados legales efectivo se estima entre  $10^{11}$  y  $10^{13}$  posiciones.

- **Complejidad del árbol de juego:** Considerando un factor de ramificación promedio  $b \approx 15$  a 20 jugadas legales por posición y una profundidad promedio de partida  $d \approx 25$  plies (medios movimientos), la complejidad de Shannon  $b^d$  para Gardner es:

$$N_G \approx 20^{25} \approx 3,3 \times 10^{32}$$

Este valor es infinitamente menor que el del ajedrez estándar ( $10^{120}$ ), lo que permite que el auto-juego converja con órdenes de magnitud menos recursos computacionales.

Dada la complejidad del entorno, nos gustaría poder modelarlo de modo que el aprendizaje se realice únicamente sobre el estado actual, ya que comprobar el historial de jugadas completo tendría un coste computacional excesivo. Buscamos que la probabilidad de alcanzar un estado futuro dependa únicamente del estado presente, y no del historial de estados anteriores. Esta es la propiedad de Markov.

### El ajedrez como Proceso de Decisión de Markov (MDP)

Un proceso estocástico cumple la propiedad de Markov si la probabilidad de alcanzar un estado futuro,  $s_{t+1}$ , depende únicamente del estado presente  $s_t$ , siendo condicionalmente independiente del historial de estados anteriores. Formalmente:

$$P(s_{t+1}|s_t, s_{t-1}, \dots, s_0) = P(s_{t+1}|s_t)$$

Si definimos el estado del ajedrez únicamente por la disposición física de las piezas en el tablero temos que  $s_t = B_t$ , entonces el juego no es estrictamente un MDP. Recordemos las reglas mencionadas anteriormente: la triple repetición y la regla de los 50 movimientos tienen en cuenta el historial de estados, no solo el actual.

Para modelar el ajedrez  $5 \times 5$  como un MDP, es necesario expandir el espacio de estados. Definimos el nuevo estado expandido  $s \in \mathcal{S}$  como una tupla  $s = (B, p, h)$  donde:

1.  $B$ : La disposición exacta de todas las piezas en el tablero de  $5 \times 5$  en el instante  $t$ .
2.  $p$ : Un reloj o contador (*half-move clock*) que registra el número de medios movimientos realizados desde la última captura o avance de peón.
3.  $h$ : Un registro de las posiciones previas ocurridas desde la última transición irreversible (o, de forma computacionalmente más eficiente, un mapa de frecuencias que contabilice cuántas veces se ha visitado la disposición  $B$  actual en la partida en curso).

Al encapsular explícitamente los contadores  $p$  y  $h$  dentro de la representación del estado actual  $s_t$ , el modelo no necesita consultar cronológicamente cómo se llegó a dicho estado. Toda la información temporal y causal necesaria para evaluar la legalidad de las jugadas futuras o detectar empates está autocontenida en el instante presente. Bajo esta formulación, se satisface la propiedad de Markov, lo que nos permite simplificar el modelado.

### 5.1.2. Arquitectura basada en Transformers: Encoder y Self-Attention

A diferencia de los modelos de lenguaje (LLMs) que generan secuencias texto *token a token* utilizando una arquitectura de decodificador con enmascaramiento causal, la evaluación de un tablero de ajedrez requiere una comprensión bidireccional del estado completo en un instante de tiempo / posición. No es necesario enmascarar la atención para evitar observar tokens futuros, ya que por la propiedad de Markov, una posición no depende de estados posteriores ni anteriores.

Por ello, la arquitectura propuesta se fundamenta en un **Transformer Encoder**. Este modelo procesa simultáneamente todas las casillas del tablero, permitiendo que cada pieza calcule su relevancia respecto a todas las demás sin restricciones de enmascaramiento. El núcleo de esta arquitectura es el mecanismo de Auto-Atención (*Self-Attention*)[12], definido formalmente como:

$$\text{Attention}(Q, K, V) = \text{softmax} \left( \frac{QK^T}{\sqrt{d_k}} \right) V$$

Donde  $Q$  (Query),  $K$  (Key) y  $V$  (Value) son matrices obtenidas mediante proyecciones lineales de las representaciones del tablero. Dada una entrada  $X$ ,

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V$$

$W^Q, W^K, W^V$  son matrices de pesos entrenables, y  $d_k$  representa la dimensión de las claves (configurada en [TODO: definir tamaño de d.k] para esta implementación). Este mecanismo se replica en cada una de las  $h$  cabezas de atención, permitiendo capturar distintas relaciones entre las piezas. El resultado se concatena y se proyecta linealmente de vuelta a la dimensión original de la entrada.

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

Donde  $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$ ,  $W^O$  es una matriz de pesos entrenables y  $h$  es el número de cabezas de atención.

### 5.1.3. Modelado del Aprendizaje por Refuerzo

El problema se modela a través de una red neuronal de política y valor conjunta  $(\mathbf{p}, v) = f_\theta(s)$ , donde  $\mathbf{p} = \pi(a|s)$  es la distribución de probabilidad sobre el espacio de acciones legales  $\mathcal{A}(s)$ , y  $v \in [-1, 1]$  es el valor escalar estimado del estado actual  $s$  (que aproxima la probabilidad de ganar desde esa posición).

Dado que el entorno de ajedrez es completamente observable y determinista, con probabilidades de transición conocidas  $P(s'|s, a) = 1$  para el único estado legal resultante, se puede implementar un algoritmo de aprendizaje por refuerzo basado en modelo (*model-based*). El agente combina la capacidad de predicción de la red neuronal con un operador de mejora de política basado en la búsqueda en árbol Monte Carlo (MCTS).

En cada paso, el MCTS realiza simulaciones guiadas por la política  $\mathbf{p}$  y el valor  $v$  estimados por la red. La política mejorada  $\pi$  obtenida de las frecuencias de visita de las ramas de búsqueda del MCTS se emplea para seleccionar la acción. El modelo se optimiza minimizando una función de pérdida multiobjetivo (estilo AlphaZero) que combina el error cuadrático medio de la predicción del valor y la entropía cruzada de la política:

$$\mathcal{L}(\theta) = (z - v)^2 - \pi^T \log \mathbf{p} + c \|\theta\|^2$$

donde  $z \in \{-1, 0, 1\}$  representa la recompensa real obtenida al finalizar la partida ( $R_T$ ), y  $c$  es un coeficiente de regularización  $L_2$  de los parámetros  $\theta$  del modelo.

### Diseño de la Función de Recompensa

El Aprendizaje por Refuerzo es altamente susceptible al fenómeno de *reward hacking*, donde el agente aprende a explotar lagunas en la función de recompensa para maximizar su puntuación sin alcanzar el objetivo real del problema. Por ejemplo, si se otorgara una recompensa positiva por cada captura de material, el agente podría aprender a retrasar el jaque mate indefinidamente para “farmear” piezas del oponente.

Para evitar este comportamiento subóptimo, se define una función de recompensa dispersa (*sparse reward*)  $R_t$ , que únicamente emite una señal al alcanzar un estado terminal final, denominado  $T$ :

$$R_t = \begin{cases} 1 & \text{si el estado } T \text{ es victoria} \\ -1 & \text{si el estado } T \text{ es derrota} \\ 0 & \text{si el estado } T \text{ es empate o no terminal} \end{cases}$$

Esta escasez de señal dificulta las fases iniciales del entrenamiento, justificando así la necesidad de técnicas avanzadas de exploración (como MCTS) o inicialización supervisada para guiar al modelo en las etapas tempranas.

#### 5.1.4. Diseño Experimental e Hipótesis

Para dar respuesta a los objetivos planteados, el desarrollo práctico se articula en torno a la demostración de las siguientes hipótesis experimentales, las cuales estructuran el diseño experimental del trabajo:

- **Hipótesis 1 (Reducción del problema):** El ajedrez 5x5 como entorno simplificado de RL puede conservar suficientes dinámicas esenciales del aprendizaje por refuerzo adversarial al tiempo que se mantiene computacionalmente manejable para la experimentación.
- **Hipótesis 2 (Representación):** El uso de representaciones estructuradas del tablero y del espacio de acciones (ej. planos espaciales vs. predicción por casillas origen/destino) mejora la eficiencia del aprendizaje frente a representaciones planas.
- **Hipótesis 3 (Inicialización):** Un agente inicializado mediante aprendizaje supervisado a partir de jugadas generadas por un motor de referencia alcanza un nivel de rendimiento competitivo en menos *epochs* que un agente entrenado de forma puramente *tabula rasa*.

### 5.2. Marco práctico

#### 5.2.1. Codificación del Espacio de Estados (Input)

Para evaluar la Hipótesis 1, se instrumentarán y compararán distintas representaciones del tablero de ajedrez 5x5, analizando su impacto en la asimilación espacial del modelo Transformer.

- **Representación plana simple (Byte Array):** El tablero se representa como un vector unidimensional de 25 *bytes* (índices 0-24), correspondientes a las 25 casillas del tablero de Minichess. Cada casilla almacena un número entero que identifica unívocamente una de las 12 piezas posibles (6 blancas y 6 negras) o el estado de casilla vacía. Esta representación minimalista es computacionalmente eficiente y delega en el Transformer la responsabilidad total de deducir la geometría 2D del tablero.
- **Representación espacial (estilo AlphaZero):** El tablero se expande en un tensor multicanal de dimensiones  $C \times 5 \times 5$ , donde cada canal  $C$  actúa como un plano binario indicando la presencia o ausencia de un tipo de pieza específico para cada color. Esta codificación facilita el reconocimiento de patrones espaciales a través de sesgos inductivos explícitos. Se añaden, además, canales binarios adicionales para variables de estado globales como el bando al que le toca mover y los límites del half-move clock.
- **Representaciones aumentadas (Heurísticas):** Se incorporarán planos adicionales que inyecten información posicional específica para guiar al Transformer. Estos planos binarios codificarán: (a) casillas atacadas por las piezas propias y del oponente, (b) piezas que se encuentran clavadas o en estado de jaque, y (c) la topología binaria del tablero (casillas blancas vs. negras) para mitigar la falta de sesgo inductivo del Transformer sobre movimientos diagonales.

#### 5.2.2. Codificación del Espacio de Acciones (Output)

De manera análoga, la forma en que la red proyecta sus predicciones altera drásticamente la topología del espacio de optimización. Se implementarán las siguientes cabezas de salida:

- **Política Plana Discreta (Flat Policy):** Consiste en un vector de salida lineal que mapea todas las posibles transiciones legales del juego. En un tablero 5x5, considerando que una pieza no puede moverse a su propia casilla origen, existen  $5^2 \times (5^2 - 1) = 600$  movimientos posicionales puros. Adicionalmente, las promociones de peón requieren un modelado independiente.

En Minichess, los peones en las columnas exteriores (a, e) tienen 2 opciones al coronar (avance o captura simple), mientras que los peones en columnas centrales (b, c, d) tienen 3. Esto resulta en  $(2 + 3 + 3 + 3 + 2) = 13$  movimientos de promoción. Multiplicado por las 4 piezas posibles de promoción (Reina, Torre, Alfil, Caballo) y los 2 colores, se añaden 104 neuronas. La salida de la política (*policy head*) será, por tanto, un vector estricto de **704 neuronas**.

- **Política Factorizada (Origen-Destino):** En lugar de predecir sobre el espacio completo de 704 acciones, la salida de la política se bifurca en dos cabezas: una cabeza que predice la casilla de origen del movimiento (vector de tamaño 25) y una segunda cabeza que predice la casilla de destino del movimiento (vector de tamaño 25). Esto reduce el espacio de salida de 704 a 50 dimensiones en total, simplificando la tarea de optimización del gradiente.
- **Cabezas auxiliares de recompensa densa:** Con el fin de mitigar la dispersión de las recompensas en las fases iniciales, se evaluará una cabeza auxiliar de valor que realice una regresión continua hacia la evaluación material o posicional (en centipeones) proporcionada en tiempo real por un motor clásico (Fairy-Stockfish) para cada estado intermedio.

### 5.2.3. Arquitectura Transformer

La arquitectura neural propuesta se basa en un modelo **Transformer Encoder** que procesa el tablero de forma bidireccional, permitiendo que cada casilla preste atención a todas las demás. El flujo de procesamiento de la red se describe a continuación:

1. **Capa de Proyección de Entrada:** Si la entrada es la representación binaria multicanal  $C \times 5 \times 5$ , se utiliza una capa convolucional de tamaño de filtro  $1 \times 1$  o una proyección lineal para mapear cada una de las 25 casillas a un vector de dimensión oculta  $d_{\text{model}}$  (generalmente configurado entre 128 y 256).
2. **Codificación Posicional Spatial 2D:** Para inyectar información geométrica y espacial en las capas de atención (ya que los Transformers puros son invariantes ante permutaciones), se suma un vector de codificación posicional 2D (con componentes seno y coseno correspondientes a las coordenadas de fila y columna de cada casilla) al embedding de cada casilla.
3. **Pila de Bloques del Encoder:** El tensor resultante de dimensiones  $25 \times d_{\text{model}}$  se procesa a través de una serie de  $L$  capas de codificación. Cada capa contiene:
  - Un mecanismo de Auto-Atención Multicabeza (*Multi-Head Self-Attention*).
  - Conexiones residuales y normalización de capa (*Layer Normalization*).
  - Una red de alimentación hacia adelante (*Feed-Forward Network*, FFN) compuesta por dos proyecciones lineales y una activación no lineal (ej. GeLU).

4. **Cabezas de Salida (Cabezas Escindidas):** Tras la última capa del codificador, el estado procesado se bifurca en dos cabezas:
  - **Cabeza de Política (*Policy Head*):** Una proyección lineal que reduce el espacio de representación a la dimensión de la política (704 salidas discretas o 50 en la versión factorizada), seguida de una capa Softmax para producir una distribución de probabilidad sobre las acciones legales del estado.
  - **Cabeza de Valor (*Value Head*):** Se aplica una capa de agregación global (*pooling*) sobre las casillas, seguida de una proyección lineal a una capa oculta, normalización, y finalmente una proyección escalar con una activación tangente hiperbólica (*tanh*) que acota la salida en el intervalo  $[-1, 1]$ .

#### 5.2.4. Algoritmo RL y Búsqueda en Árbol Monte Carlo (MCTS)

La actualización del modelo se basa en un bucle iterativo de auto-juego guiado por MCTS. Cada nodo en la búsqueda representa un estado del tablero  $s$ , y cada rama representa una jugada legal  $a \in \mathcal{A}(s)$ . Durante la búsqueda MCTS, cada arista  $(s, a)$  almacena cuatro estadísticas clave:

- $N(s, a)$ : Número de visitas o simulaciones acumuladas por la rama.
- $W(s, a)$ : Valor total de la acción acumulado de las simulaciones.
- $Q(s, a)$ : Valor medio de la acción, calculado como  $Q(s, a) = W(s, a)/N(s, a)$ .
- $P(s, a)$ : Probabilidad a priori de seleccionar la acción, obtenida de la red de política:  $P(s, a) = \mathbf{p}(a|s)$ .

El algoritmo MCTS se ejecuta durante un número fijo de iteraciones por cada movimiento en la partida, constando de cuatro fases cíclicas:

1. **Selección:** Partiendo del nodo raíz (estado actual), el algoritmo desciende seleccionando la acción que maximiza el límite superior de confianza para árboles (*PUCT - Predictor Upper Confidence Trees*):

$$a_t = \arg \max_a \left( Q(s, a) + c_{\text{puct}} P(s, a) \frac{\sqrt{\sum_b N(s, b)}}{1 + N(s, a)} \right)$$

donde  $c_{\text{puct}}$  es una constante hiperparámetro que regula el balance entre exploración y explotación.

2. **Expansión y Evaluación:** Al alcanzar un nodo hoja  $s_L$  (un estado no explorado previamente en la búsqueda), este es evaluado por la red de política-valor de forma asíncrona:  $(\mathbf{p}, v) = f_\theta(s_L)$ . Las acciones legales en  $s_L$  se expanden y se inicializan sus prioridades a  $P(s_L, a) = \mathbf{p}(a)$ .
3. **Retropropagación:** El valor  $v$  devuelto por la red para el nodo hoja se propaga hacia arriba en el árbol. Para cada arista visitada  $(s, a)$  en la simulación:

$$N(s, a) \leftarrow N(s, a) + 1, \quad W(s, a) \leftarrow W(s, a) + v$$

recalculándose el promedio  $Q(s, a)$  en consecuencia.



4. **Decisión final:** Una vez concluidas las simulaciones, la política de búsqueda final del MCTS para la selección del movimiento en la partida real se define como:

$$\pi(a|s_0) = \frac{N(s_0, a)^{1/\tau}}{\sum_b N(s_0, b)^{1/\tau}}$$

donde  $\tau$  es un parámetro de temperatura de exploración.

### 5.2.5. Herramientas y tecnologías

El desarrollo práctico e investigativo del proyecto se implementará utilizando las siguientes herramientas de desarrollo de software libre:

- **Lenguaje de programación:** Python 3.10+ por su extenso soporte y bibliotecas nativas de Deep Learning.
- **Framework de Deep Learning:** PyTorch para el diseño de la red Transformer, cálculo automático de gradientes y aceleración en GPU mediante CUDA.
- **Motor de Ajedrez Auxiliar:** Se emplea **Fairy-Stockfish** (versión compilada para variantes personalizadas) a través de la API **pyffish** de Python. Este motor clásico nos proporciona: (a) validación robusta y rápida de la legalidad de movimientos y reglas de la variante Gardner  $5 \times 5$ , y (b) evaluaciones heurísticas de precisión para inicializar las cabezas auxiliares densas.
- **python-chess:** Biblioteca complementaria para la gestión del tablero en Python, serialización de partidas en formato PGN y manipulación de cadenas FEN.
- **Seguimiento de experimentos:** **Weights & Biases** (wandb) para la visualización en tiempo real del progreso de entrenamiento, entropía de la política, pérdida multiobjetivo y curvas de convergencia.

## 6. Planificación y seguimiento

La planificación del trabajo es una aproximación orientada a la experimentación rápida. Para minimizar la fricción en el desarrollo de los experimentos y la lógica interna del ajedrez Gardner (validación de jugadas, etc.) se prescinde del desarrollo de un motor de *bitboards* desde cero, delegando la gestión del entorno y la generación de datos en la herramienta **Fairy-Stockfish** a través de su interfaz en Python, **pyffish**. Esta decisión acelera las fases iniciales del proyecto y permite además obtener datos de buena calidad para el preentrenamiento de los modelos. A continuación se presenta el plan de trabajo, dividido en fases:

### Fase 0: Investigación previa y documentación inicial

- Investigación del estado del arte en RL y Transformers aplicados al ajedrez.
- Consolidación de objetivos generales y específicos. Creación del marco teórico y práctico para el desarrollo del TFG.

### Fase 1: Preparación del Entorno y Generación de Datos Sintéticos

- Integración de **pyffish** para la configuración de la variante **gardner** y validación de movimientos.
- Desarrollo y ejecución de *scripts* de *self-play* basados en evaluaciones a profundidad controlada de Fairy-Stockfish para construir el *dataset* de conocimiento inicial.

### Fase 2: Arquitectura y Preentrenamiento (Supervisado)

- Implementación del modelo Transformer Encoder y las distintas capas de adaptación (I/O).
- Ejecución de experimentos de aprendizaje supervisado utilizando los *datasets* generados, evaluando la velocidad de convergencia de las distintas representaciones de tablero y acción.

### Fase 3: Aprendizaje por Refuerzo (Pipeline de Self-Play)

- Construcción del bucle MCTS (Monte Carlo Tree Search) guiado por las predicciones de política y valor del Transformer.
- Implementación del algoritmo de actualización de política (ej. PPO).

### Fase 4: Evaluación y Generación de Métricas

- Ejecución de torneos automatizados entre agentes de control (Stockfish limitado, modelos puramente *tabula rasa* frente a modelos preentrenados).
- Extracción y visualización de las métricas de rendimiento para la validación de las hipótesis planteadas.

### Fase 5: Documentación y Conclusiones

- Consolidación de los resultados en el documento de Trabajo de Fin de Grado, análisis crítico de las gráficas resultantes y preparación de la defensa.

## 7. Principales aportaciones

TODO aquí las gráficas!!!

- No introducir conceptos nuevos. directo al grano.
- Agrupar los resultados por cada una de las 3 hipótesis. Insertar las 1 o 2 gráficas correspondientes a cada bloque.

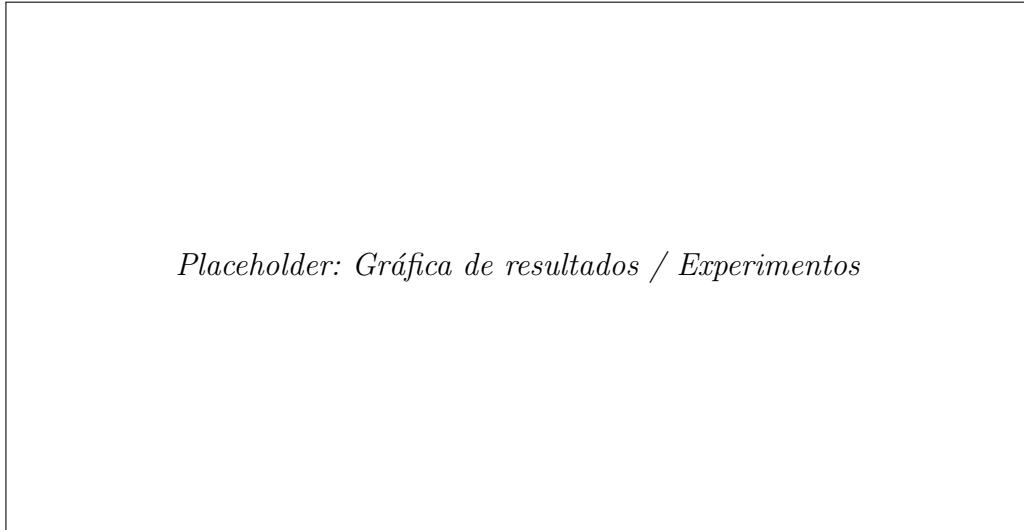


Figura 2: Descripción del experimento y métricas obtenidas.

## 8. Conclusiones

TODO

- Análisis crítico de las gráficas, lectura matemática.
- ¿Se confirman las hipótesis o se desmienten?

## 9. Vías de trabajo futuro

TODO Lo que me quedó en el tintero

## Referencias

- [1] R. S. Sutton. “The Bitter Lesson.” (2019), URL: <http://www.incompleteideas.net/IncIdeas/BitterLesson.html>.
- [2] D. Silver, A. Huang, C. J. Maddison et al., “Mastering the game of Go with deep neural networks and tree search,” *Nature*, vol. 529, n.º 7587, pp. 484–489, 2016. DOI: 10.1038/nature16961. URL: <https://www.nature.com/articles/nature16961>.
- [3] D. Silver, T. Hubert, J. Schrittwieser et al., “A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play,” *Science*, vol. 362, n.º 6419, pp. 1140–1144, 2018. DOI: 10.1126/science.aar6404. URL: <https://www.science.org/doi/10.1126/science.aar6404>.
- [4] J. Jumper, R. Evans, A. Pritzel et al., “Highly accurate protein structure prediction with AlphaFold,” *Nature*, vol. 596, n.º 7873, pp. 583–589, 2021. URL: <https://www.nature.com/articles/s41586-021-03819-2>.
- [5] Y. Nasu, “Efficiently Updatable Neural-Network-based Evaluation Functions for Computer Shogi,” en *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018, pp. 1–7. URL: <https://github.com/asdfjkl/nnue>.
- [6] A. Ruoss, G. Delétang, S. Medapati et al., *Amortized Planning with Large-Scale Transformers: A Case Study on Chess*, 2024. arXiv: 2402.04494 [cs.LG]. URL: <https://arxiv.org/abs/2402.04494>.
- [7] D. Monroe e P. A. Chalmers, *Mastering Chess with a Transformer Model*, 2024. arXiv: 2409.12272 [cs.LG]. URL: <https://arxiv.org/abs/2409.12272>.
- [8] D. Monroe, G. Eilender, P. Chalmers, Z. Tang e A. Anderson, *Chessformer: A Unified Architecture for Chess Modeling*, 2026. arXiv: 2605.19091 [cs.LG]. URL: <https://arxiv.org/abs/2605.19091>.
- [9] E. Jenner, S. Kapur, V. Georgiev, C. Allen, S. Emmons e S. Russell, *Evidence of Learned Look-Ahead in a Chess-Playing Neural Network*, 2024. arXiv: 2406.00877 [cs.LG]. URL: <https://arxiv.org/abs/2406.00877>.
- [10] Daniel Monroe. “Transformer Progress in Leela Zero.” (2024), URL: <https://lczero.org/blog/2024/02/transformer-progress/>.
- [11] C. E. Shannon, “Programming a Computer for Playing Chess,” *Philosophical Magazine*, vol. 41, n.º 314, pp. 304–317, 1950.
- [12] A. Vaswani, N. Shazeer, N. Parmar et al., “Attention is All You Need,” en *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 5998–6008.

## A. Anexos

Cosas a meter:

- instrucciones reproducibilidad: configuración del entorno, los comandos de consola para lanzar los scripts de entrenamiento y generación de datos...
- pseudocódigos completos
- tablas de hiperparámetros wandb